DATOLOGYAI RESEARCH

The Finetuner's Fallacy

When to Pretrain with Your Finetuning Data

A Comprehensive Analysis of Specialized Pretraining

TABLE OF CONTENTS

01

EXECUTIVE SUMMARY

Key findings and breakthrough insights from the research

RESEARCH OVERVIEW

Core Contributions & Key Findings

The Problem

Real-world deployments demand strong performance on **narrow domains** where data is often scarce. Standard practice finetunes models, but this risks **overfitting** and **forgetting**.

The Solution

Specialized Pretraining (SPT): Mix domain data into pretraining as a **small fraction δ** of tokens, typically repeating **10-50×** over training.

Key Results

Token Reduction Fewer pretraining tokens needed	1.75×
1B vs 3B Model Gap closed on far-from-web domains	133%
MATH Accuracy Improvement at 200B tokens	+6%

★ SPT improves domain performance AND preserves general capabilities

The Finetuner's Fallacy: Finetuning appears cheapest but early integration of domain data yields better results with fewer parameters and less total compute when amortized over inference.

KEY FINDINGS

Major Discoveries at a Glance

1 Consistent Performance Gains

SPT improves post-finetuning performance across **chemistry, music, and mathematical proofs** domains.

2.0%	1.5%	0.8%
MusicPile	ProofPile	ChemPile

2 Training Efficiency

SPT reduces pretraining tokens needed to reach target performance by **1.4-1.75×** depending on domain.

1.75×	1.56×	1.40×
MusicPile	ProofPile	ChemPile

3 Parameter Efficiency

On domains far from web text, **smaller SPT models outperform larger standard models.**

1B SPT > 3B NPT

On ProofPile (133% gap closure)

4 Early Integration Wins

Early integration of domain data yields better results than reserving it all for finetuning. **When data appears matters as much as whether it appears.**

→ **Incorporate domain data as early as possible**

02

INTRODUCTION

Understanding the conventional approach and its limitations

CONVENTIONAL APPROACH

The Standard Pipeline: Naive Pretraining → Finetuning

The Scenario

Consider an organization with **proprietary data** such as:

- Support Conversations
- Legal Filings
- Clinical Notes
- Research Data

The goal: Train a **domain-specialized model** that excels on this proprietary content.

The Recipe

1. **Start Strong**
   Use open-weights model pretrained on web-scale data

↓

2. **Finetune**
   Train on proprietary dataset to inject domain knowledge

The Reinforced View

Success of **instruction tuning, RLHF, and parameter-efficient finetuning** has reinforced this pipeline view.

Why This Seems Right

Because domain data is **private and absent** from public corpora, finetuning is treated as the natural mechanism for injecting missing domain knowledge.

Modern training pipelines treat pretraining and finetuning as disjoint phases: First learn general knowledge at scale, then specialize using a small curated dataset.

EMERGING EVIDENCE

When Data Appears Matters More Than We Thought

Growing evidence suggests that **data encountered during pretraining shapes model behavior more durably** than data introduced later:

Reasoning Data

Incorporating reasoning data into **pretraining** outperforms introducing it only during **finetuning**

Source: Akter et al., 2025; Hatamizadeh et al., 2025

Safety Behaviors

Unsafe behaviors learned during pretraining are **harder to remove** via post-training interventions

Source: Maini et al., 2025; Sam et al., 2025

Multilingual Transfer

Cross-language transfer during pretraining improves low-resource language performance more effectively

Source: Longpre et al., 2025

The Unexplored Question

Yet the question of **when to introduce domain-specific data**, and whether it should be mixed into pretraining rather than reserved for finetuning, remains largely unexplored.

RESEARCH QUESTION

Challenging the Conventional Wisdom

The Core Question

Is **reserving all domain-specific data** for the final stage of training optimal?

When target domain is **poorly represented** in pretraining corpus, introducing domain data only during finetuning may require:

- Large representational updates
- Weaker generalization
- Greater forgetting of general knowledge

Specialized Pretraining (SPT)

1 Mix During Pretraining

Domain data as **small fraction δ** of tokens, often repeating **10-50×**

+

2 Finetune as Usual

Then finetune on the **same domain data** as standard practice

The Alternative

Study a simple alternative: **interleave domain dataset throughout pretraining** as a small fraction of training tokens

Key Insight: Interleaving domain tokens with general data allows the model to tolerate far more repetitions before overfitting than during finetuning

03

SPT METHODOLOGY

Understanding the Specialized Pretraining approach and experimental framework

DEFINITION

What is Specialized Pretraining (SPT)?

Definition

A training strategy where **domain-specific data is mixed into pretraining** as a small fraction δ of total tokens, typically repeated **10-50×** over the course of training.

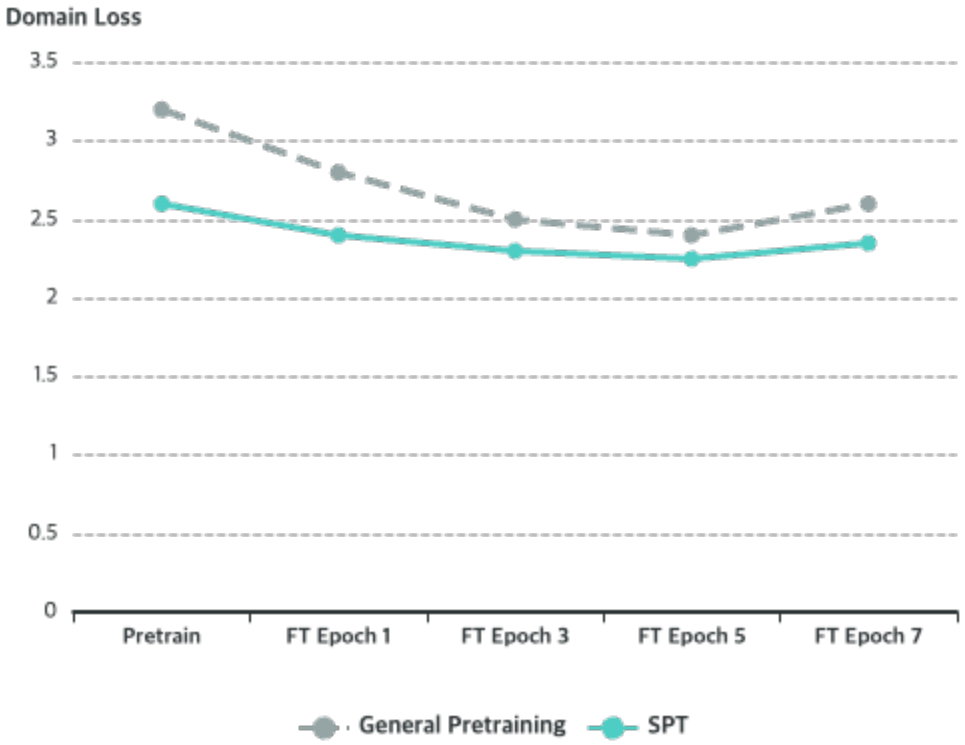
SPT → FT Pipeline

Pretrain with domain mix → Finetune on domain

Key Insight

Interleaving domain tokens with general data allows the model to **tolerate far more repetitions before overfitting** than it would during finetuning alone.

SPT vs General Pretraining



Domain Test Loss
Lower with SPT

General Knowledge
Less forgetting

NOTATION

Key Notation & Definitions

Core Notation

δ Mixture Fraction

Fraction of pretraining tokens from domain dataset: $\delta \in [0, 1]$

Example: $\delta = 0.02 = 2\%$ domain tokens

E Domain Epochs

Total epochs of domain data seen: $E = (T \cdot \delta) / |D_{dom}|$

T = pretraining tokens, $|D_{dom}|$ = domain dataset size

Training Paradigms

NPT Naive Pretraining

Standard pretraining on **general web data only** ($\delta = 0$)

↓

SPT Specialized Pretraining

Pretraining with **domain data mixed in** ($\delta > 0$)

+

FT Finetuning

Both followed by **finetuning on domain data**

Test Split

A **fixed test split** is held out from each domain dataset and **never included in training.** All domain losses reported are evaluated on this held-out test split.

Relative Gain: $R_{gain}(\delta) = 100 \cdot (L_{NPT \to FT} - L_{SPT(\delta) \to FT}) / L_{NPT \to FT}$

EXPERIMENTAL SETUP

OLMo Sandbox: Controlled Environment

Base Configuration

Base Model
OLMo-1B architecture

Pretraining Corpus
Dolma dataset

Pretraining Tokens
200B tokens

Mixture Fractions
$\delta \in \{0, 0.1\%, 1\%, 2\%, 5\%\}$

Example: 5% SPT repeats domain-specific data roughly **33×** over course of pretraining

Three Domains

MusicPile
Symbolic music notation
~300M tokens

ChemPile
Chemistry literature
~300M tokens

ProofPile
Mathematical proofs
~300M tokens

Matched Settings

Match OLMo-1B pretraining settings: **optimizer, cosine learning rate schedule, batch size.** All hyperparameters documented in Appendix B.

Each domain: ~300M tokens

MODEL VARIANTS & FT

Model Size Variants & Finetuning Protocol

Model Size Variants

Create variants by **adjusting model depth:**

300M — **Halve layers**
Reduce MLP & hidden dims

600M — **Halve layers**
Reduce MLP & hidden dims

1B — **Base OLMo-1B**
Reference architecture

3B — **Double layers**
From 1B architecture

All models use same pretraining recipe

Finetuning Protocol

Training Data

Exclusively on **domain-specific dataset** with WSD learning rate schedule

No Repetition Restrictions

Early stopping applied: data repeated as long as test loss decreases

Hyperparameter Tuning

Grid search for warmup steps and learning rate

Reporting

Report **lowest domain test loss** across all finetuning configurations

Fair comparison: Any SPT improvement is NOT from additional passes over data

04

CORE RESULTS

Performance improvements across domains and metrics

COMPARISON FRAMEWORK

SPT vs NPT: The Comparison Framework

Two Training Paradigms

NPT→FT

Naive Pretraining → Finetuning

- ✓ Pretrain on Dolma for 200B tokens (δ = 0)
- ✓ Then finetune on domain data

SPT→FT

Specialized Pretraining → Finetuning

- ✓ Pretrain on domain repeats mixed with Dolma for 200B tokens
- ✓ Then finetune on same domain data

Relative Gain Formula

$$R_{gain}(\delta) = 100 \cdot (L_{NPT \to FT} - L_{SPT(\delta) \to FT}) / L_{NPT \to FT}$$

- L **Domain test loss** for each paradigm
- δ **Mixture fraction** for SPT
- R **Relative gain** as percentage improvement

Key Question

Which paradigm achieves **lower domain test loss** after finetuning?

Interpretation

Higher R_{gain} means **SPT outperforms NPT** by a larger margin. Positive values indicate SPT is better.

PERFORMANCE RESULTS

Domain Test Loss Improvements

SPT **consistently helps across all three domains**, with gains varying based on domain similarity to web text:

MusicPile

2.0%

Relative Gain

Symbolic music notation that bears little resemblance to web text

Furthest from web text → Largest gain

ProofPile

1.5%

Relative Gain

Formal mathematical proofs with specialized notation and structure

Second furthest from web text

ChemPile

0.8%

Relative Gain

Chemistry text closer to Dolma distribution (more natural language)

Closest to web text → Modest gain

Pattern: The further the domain is from web text, the larger the benefit from SPT

COMPUTE EFFICIENCY

Training Efficiency: Compute Multiplier

Compute Multiplier Definition

The **factor by which SPT reduces** the number of pretraining tokens needed to reach a given post-finetuning domain test loss, compared to NPT.

Compute Multiplier

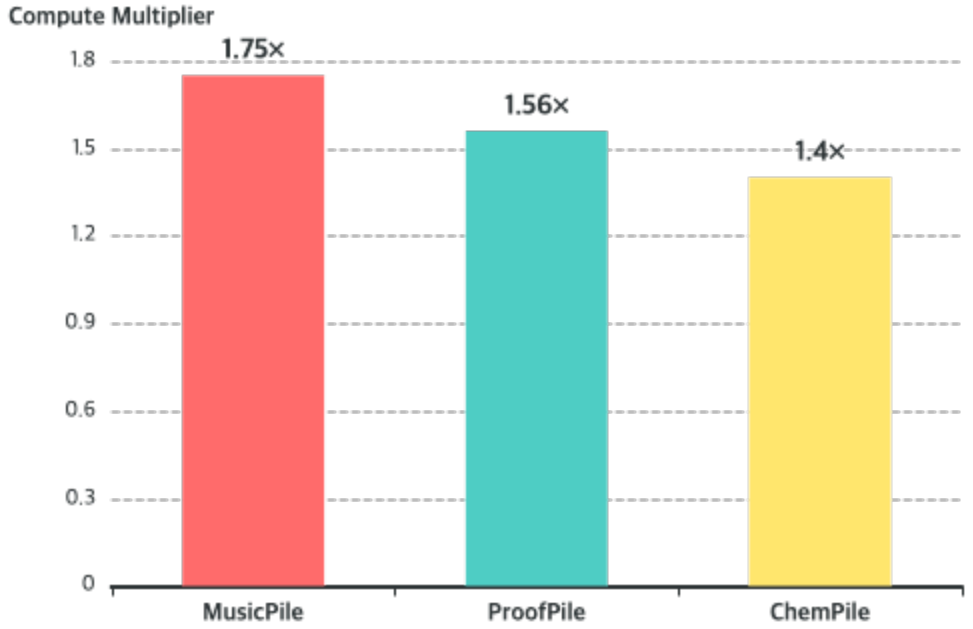
$= \text{Tokens}_{\text{NPT}} / \text{Tokens}_{\text{SPT}}$

Higher = More efficient

Key Insight

Efficiency gains are **substantial** across all domains, meaning SPT reaches target performance faster.

Efficiency Results



1.75×	1.56×	1.40×
MusicPile	**ProofPile**	**ChemPile**
~114B vs 200B	~128B vs 200B	~143B vs 200B

PARAMETER EFFICIENCY

Can SPT Compensate for Model Size?

The Critical Question

If a **smaller model trained with SPT** could match or exceed the performance of a **larger NPT model**, the practical implications would be substantial.

Test Setup:

- ✓ Pretrain 3B model on 200B Dolma tokens (NPT)
- ✓ Measure gap closure by 1B SPT model

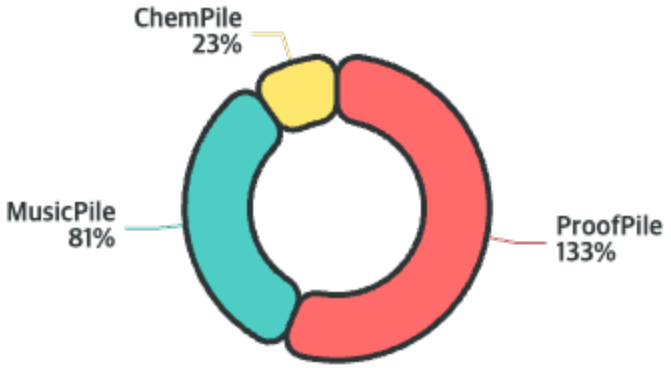
Gap Closure Formula:

% of (1B NPT → 3B NPT) gap closed by 1B SPT

>100% means 1B SPT surpasses 3B NPT

SPT delivers better domain performance, faster convergence, and stronger parameter efficiency

Gap Closure Results



ProofPile — 133%

1B SPT **surpasses** 3B NPT entirely

MusicPile — 81%

Nearly **matches** the larger model

ChemPile — 23%

More **modest** gains (consistent with smaller R_{gain})

DOWNSTREAM TASKS

Downstream Task Performance

Loss improvements **translate to downstream accuracy** on domain-matched benchmarks:

MusicPile → MusicTheoryBench

Accuracy Improvement

+4%

at 200B tokens

Symbolic music questions in 4-choice MCQA format

2% SPT vs NPT baseline

ChemPile → ChemBench

General Chemistry Subset

↑

Consistent improvements

Chemistry knowledge and reasoning tasks

Across most pretraining budgets

ProofPile → MATH

Accuracy Improvement

+6%

at 200B tokens

Mathematical reasoning in MCQA style

2% SPT vs NPT baseline

✓ Across all benchmarks and most pretraining budgets, SPT outperforms NPT by several percentage points

05

LEARNING & FORGETTING

Understanding the dual benefits of specialized pretraining

DUAL BENEFIT #1

Reduced Forgetting of General Knowledge

During Pretraining

SPT allocates a **small fraction** of pretraining tokens to domain data, but this has **minimal impact** on Dolma loss:

Comparable General Loss

NPT and SPT runs achieve **comparable general loss** after 200B tokens

The Difference Emerges During Finetuning

This is where SPT's advantage becomes clear.

During Finetuning

NPT

- ↑ Starts with **higher domain loss**
- ↑ Requires **more aggressive optimization**
- ↑ **Drives up general loss** (forgetting)

≠

SPT

- ↓ Starts from **lower domain loss**
- ↓ Requires **less adaptation**
- ↓ Exhibits **less forgetting**

DUAL BENEFIT

The Dual Benefit: Domain + General Performance

Correlation with Downstream Tasks

The **combination of lower domain loss and lower general loss** correlates with improved downstream task performance.

The Connection

Better domain understanding + preserved general knowledge = Better task performance

SPT's Key Advantage

Retaining **both general and domain performance simultaneously** is what makes SPT practically attractive.

How SPT Achieves This

1 During Pretraining

Domain data introduced as **small fraction alongside general web data**

→ Model learns domain structure **without sacrificing broad coverage**

↓

2 During Finetuning

Translates into **less catastrophic forgetting**

→ SPT models start from lower domain loss, require less aggressive adaptation, **preserving general capabilities**

This dual benefit underlies the finetuner's fallacy

06

OVERFITTING MECHANISM

Why SPT works: The regularization effect

THE CHALLENGE

The Overfitting Challenge in Domain Adaptation

Prior Work Context

Prior work on mixing data during pretraining typically studies regimes where:

Typical Setting:

- ✓ Data is **abundant**
- ✓ Rarely **repeated**
- ✓ Mixing primarily **accelerates optimization**

Our Setting Differs:

Finetuning datasets are **at least an order of magnitude smaller** than model size:

300M tokens VS 1B parameters

The Overfitting Risk

With reasonable repetitions, models **can overfit** to the training data:

Test Loss Rises

Across domains, for both SPT→FT and NPT→FT, domain test loss **begins to rise after ~5 epochs** of finetuning

Key Insight

SPT's benefit is better understood as a **regularization effect** rather than an optimization one

SPT reduces overfitting through diffused exposure during pretraining

THE MECHANISM

How SPT Reduces Overfitting: The Mechanism

During Pretraining

Small Fraction

Domain tokens make up only a **small fraction of each batch**

Natural Regularizer

Surrounding **general data acts as natural regularizer**, preventing memorization even after tens of repetitions

Result

Model can tolerate **far more repetitions** before overfitting

→ Pretraining: Low overfitting risk

During Finetuning

Regularization Absent

This regularization effect is **absent:** model trains exclusively on domain data

Rapid Overfitting

Model **overfits rapidly** without the protective mixing of general data

Train-Test Gap Widens

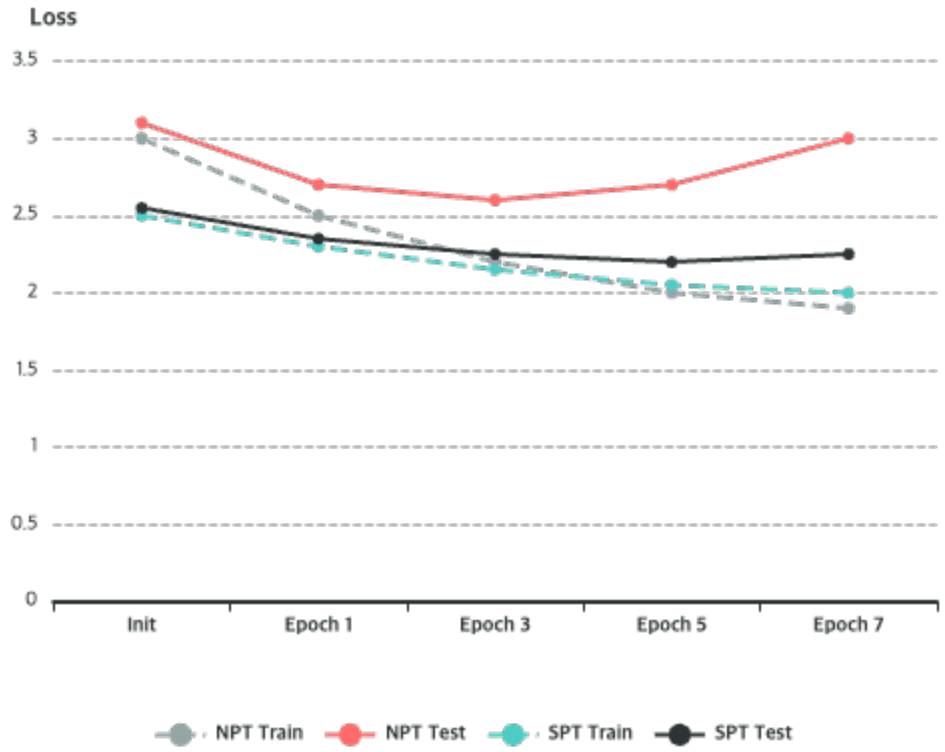
At same training loss, NPT model's **train-test gap widens much faster** than SPT

→ Finetuning: High overfitting risk

EMPIRICAL EVIDENCE

Evidence: Train–Test Gap Analysis

Train-Test Gap Evolution



NPT
Gap widens fast

SPT
Gap widens slow

Key Observations

1 Early Stage

At same domain training loss, comparing SPT at initial pretrained checkpoint with NPT after early finetuning steps, **both models generalize comparably**

2 As Finetuning Continues

NPT model's **train-test gap widens much faster** than SPT model

3 SPT Advantage

SPT models enter finetuning with **lower domain loss**, need less adaptation, and **exit before overfitting** sets in

★ Even δ=5% model (33× repeats) overfits less than NPT seeing data for first time

07

KEY FACTORS

When does SPT work best? Four key factors

FACTOR 1

Domain Similarity

The Core Principle

The **degree of similarity** between domain-specific data and general pretraining corpus influences SPT efficacy.

Limiting Case:

Where pretraining and target distributions **coincide**, incorporating domain data during pretraining introduces **no additional signal** and shouldn't improve performance.

Our Question:

How does **distributional overlap** affect SPT's relative gain?

Two Approaches

We examine this through **two complementary approaches:**

Approach 1: Controlled Study

Systematic Variation

Overlap is **systematically varied** in a controlled setting

- ✓ English→Japanese translation task
- ✓ Vary Japanese % in pretraining mix
- ✓ Measure R_{gain} at each level

Approach 2: Cross-Domain Analysis

Natural Distribution Shifts

Correlation analysis across **naturally occurring** distribution shifts

- ✓ Compare MusicPile, ChemPile, ProofPile
- ✓ Use distributional similarity metrics
- ✓ Correlate with observed R_{gain}

CONTROLLED STUDY

English→Japanese Translation Study

Why Translation?

This task is **well studied** and presents **substantial linguistic divergence:**

- **Script**
- **Morphology**
- **Word Order**

Standard Practice:
Pretrain on **monolingual corpora** from source and target languages, then finetune on **parallel data**

Manipulation

Vary proportion of **Japanese relative to English** monolingual text during pretraining to systematically alter overlap

Experimental Design

Models

160M-parameter LLaMA models

Pretraining Data

20B tokens FineWeb2 English + Japanese monolingual
1B tokens parallel data from JParaCrawl v3.0

SPT Mixtures

$\delta \in \{0\%, 0.1\%, 1\%, 5\%\}$ from parallel corpus

Japanese Variation

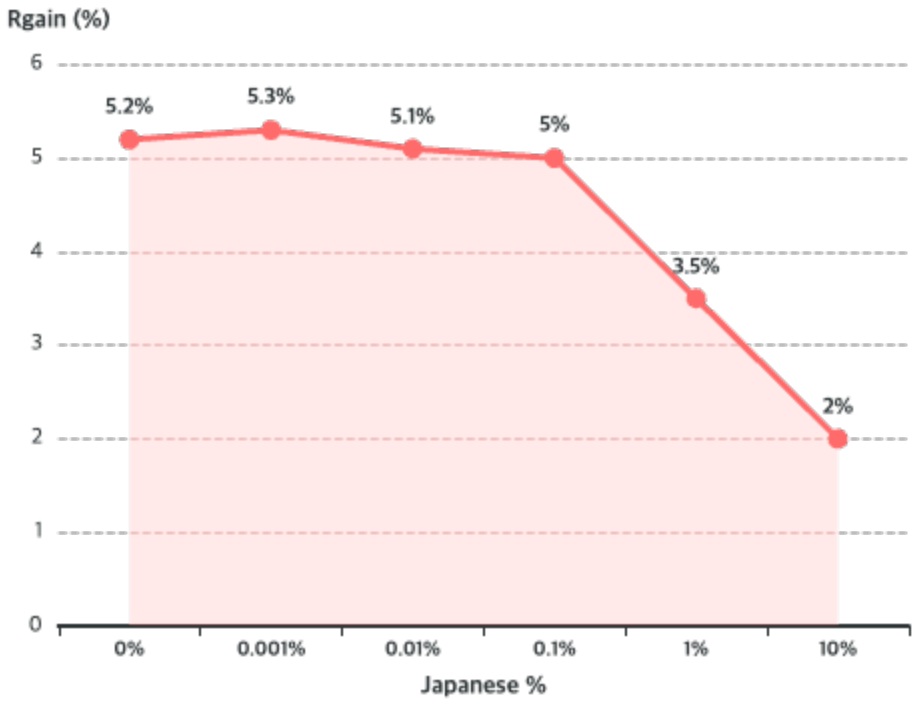
Japanese %: **{0%, 0.001%, 0.01%, 0.1%, 1%, 10%}**

All models finetuned on parallel corpus

RESULTS

Translation Study: Results

Rgain vs Japanese %



10% Japanese	0.1% Japanese
~2%	~5%

Key Findings

1 Inverse Relationship

As Japanese proportion **decreases from 10% to 0.1%**, R_{gain} **increases from ~2% to ~5%**

2 Plateau Effect

R_{gain} **plateaus at approximately 5%** beyond 0.1% Japanese

3 Diminishing Returns

More Japanese monolingual content **reduces distributional gap**, diminishing marginal benefit of parallel data

✓ Controlled evidence: R_{gain} driven by distributional misalignment

KEY INSIGHT

Translation Study: key Insight

The Mechanism

More Japanese

Increasing Japanese monolingual content during pretraining **reduces distributional gap** between pretraining and finetuning (parallel translation) data

↓

Diminished Benefit

This **diminishes the marginal benefit** of including parallel data early

Controlled Evidence

These findings provide **controlled evidence** that relative gain from SPT is driven by **distributional misalignment**

The Core Principle

When pretraining corpus already contains domain-relevant signal:

↓

Smaller R_{gain}

When domain is underrepresented:

↑

Larger R_{gain}

SPT most beneficial when target domain is far from pretraining distribution

CROSS-DOMAIN ANALYSIS

Cross-Domain Correlation Analysis

Research Question

Can standard measures of **distributional similarity** between Dolma and each specialized domain predict R_{gain}?

Five Metrics Tested:

1. Unigram JSD
2. Bigram JSD
3. Trigram JSD
4. MAUVE
5. C2ST

Direct Proxy

Also use: **Domain loss of NPT checkpoints after finetuning** (higher loss = less overlap)

Results

Japanese Sweep

All metrics correlate **strongly** with R_{gain} ($|r| > 0.85$)

Cross-Domain

Metrics **disagree** on which domain is closest to Dolma

Only One Winner

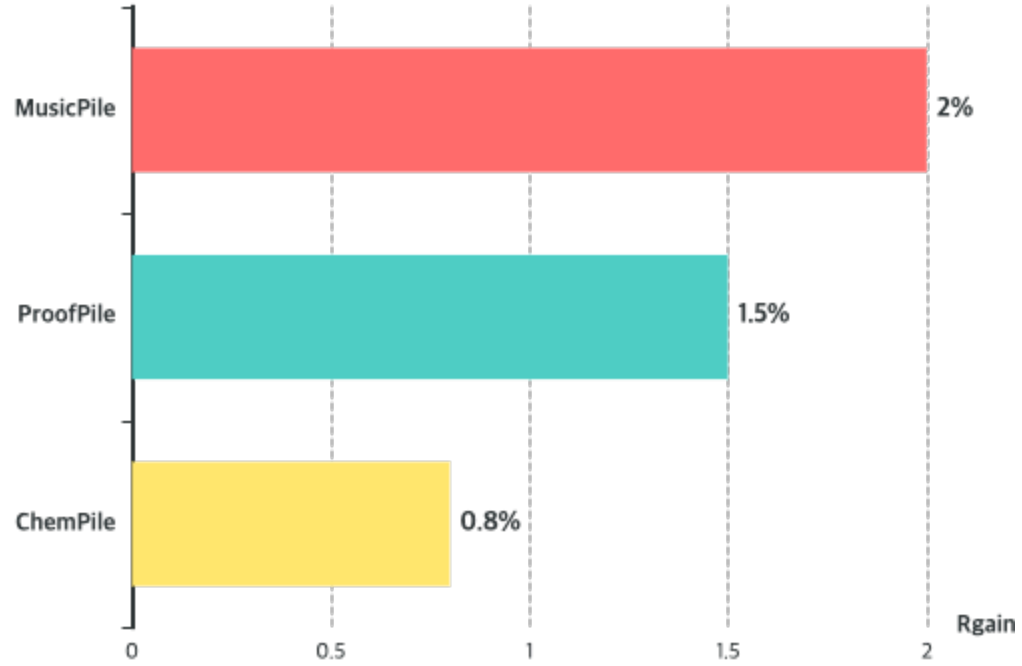
Post-finetuning domain loss correctly ranks all three domains

Direct measure of generalization beats distributional metrics

METRIC ANALYSIS

Cross-Domain: Metric Performance

Known Rgain Ranking



Higher bar = More benefit from SPT

Metric Performance

Unigram JSD	Weak (r=0.32)
Bigram JSD	Weak (r=0.44)
Trigram JSD	Better (r=0.57)
MAUVE	Weak (r=-0.23)
C2ST AUC	WRONG (r=-0.98)
Post-FT Loss ✓	CORRECT

Only post-finetuning domain loss correctly ranks all three domains

FACTOR 2

Domain Dataset Size

The Overfitting Risk

Even when overlap between pretraining and target data is substantial, **repeated exposure to limited domain corpus** can induce overfitting.

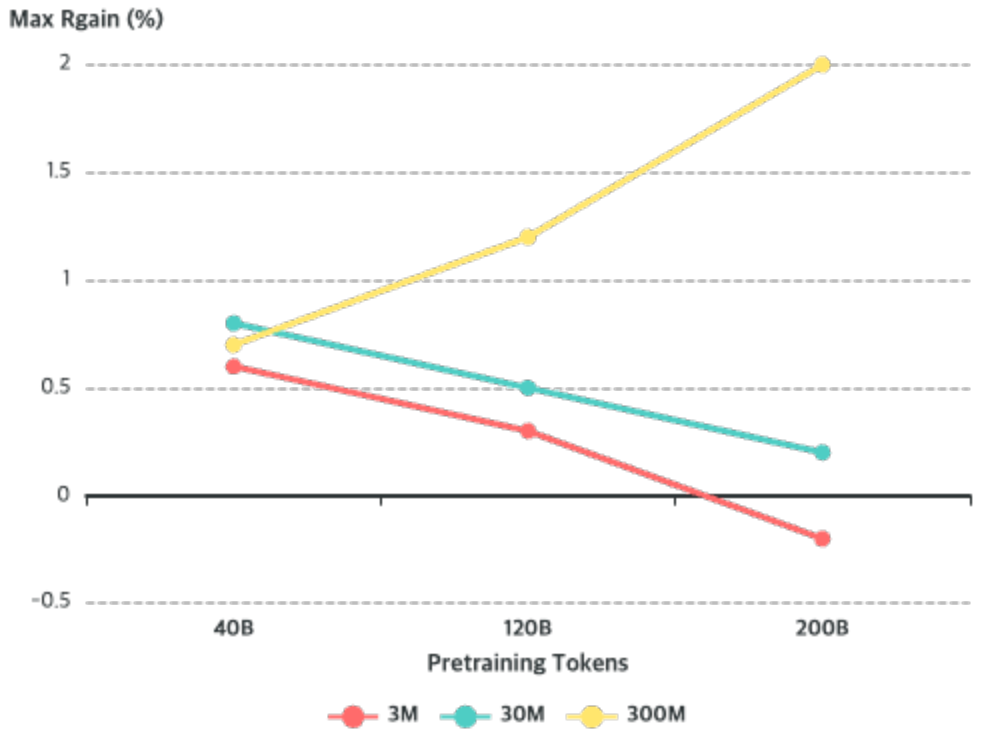
Research Question:

How does magnitude of SPT's relative gain vary as function of **domain dataset size**?

Experimental Setup

Evaluate mixture fractions $\delta \in \{0, 0.1, 1, 2, 5\}$ across subsets of **{3M, 30M, 300M}** tokens from MusicPile

Maximum R_{gain} by Dataset Size



3M tokens	30M tokens	300M tokens
Small	Medium	Large

SCALING REGIMES

Dataset Size Results: Scaling Regimes

40B

Modest Scale

Results:

- ✓ SPT yields **consistent gains**
- ✓ $R_{gain} \geq 0.5\%$ **across all sizes**
- ⭐ **30M subset** shows largest improvement

All sizes benefit at 40B

120B

Medium Scale

Results:

- ↑ **300M:** Gain continues increasing
- ↓ **3M-30M:** Benefit diminishes
- ✕ Can become **negative**

Regimes begin to diverge

200B

Large Scale

Results:

- ↑ **300M:** Gain continues increasing
- ⚠ **Small datasets:** Excessive repetition
- ✕ Leads to **overfitting** and degradation

Clear separation of regimes

Key Insight: Small mixture fractions (e.g., δ=0.1%) induce excessive repetition on small datasets, leading to overfitting during pretraining

ALTERNATIVE APPROACH

Specialized Continued Pretraining (SCPT)

The Problem

In **extremely data-constrained regimes** (tens of millions of tokens or fewer), even small mixture fractions can lead to excessive overfitting.

The Question:

Is there a better approach for very small domain datasets?

The Solution: SCPT

Specialized Continued Pretraining: Introduce domain data at **later stages** of pretraining rather than from the beginning

SCPT Implementation

1 Start with NPT

Train **180B tokens** with standard naive pretraining ($\delta = 0$)

↓

2 Continue with Domain Mix

Continue training for **additional 20B tokens** with 1% domain mixture

Results

30M dataset: SCPT achieves **higher Rgain** than full SPT

3M dataset: SCPT at 1% **still overfits** excessively

★ **Key Insight: Interaction between repetition and dataset size induces distinct scaling regimes**

FACTOR 3

Model Size

Key Finding

Relative gains from SPT **persist across model scales** and actually **increase with parameter count**.

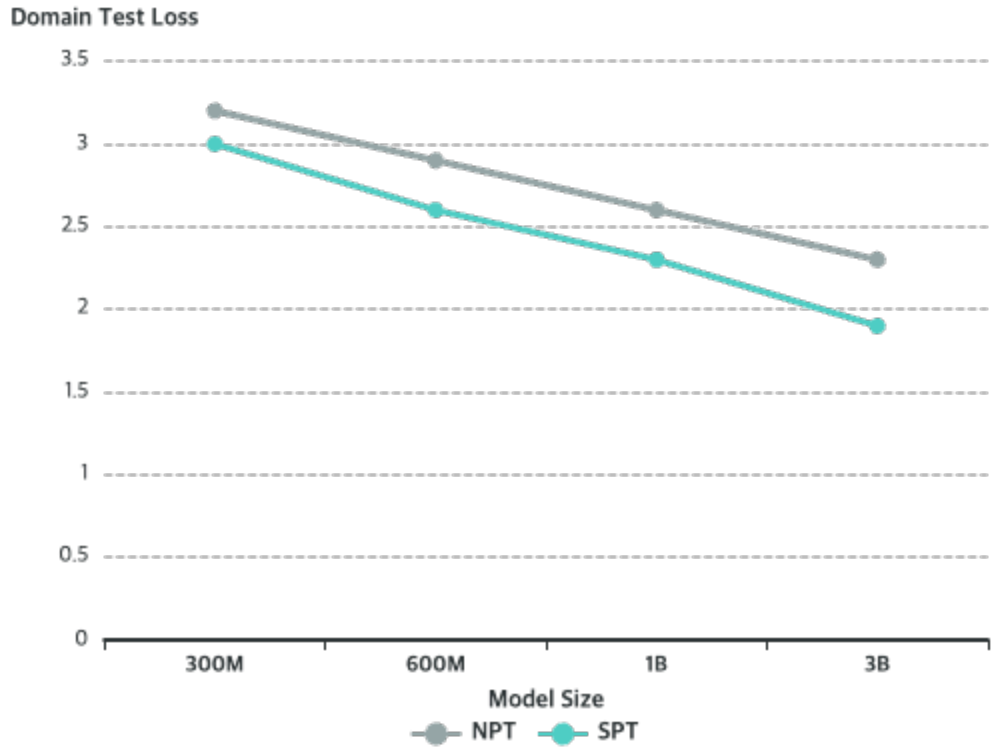
Observation:

Gap in test loss between SPT and NPT **widens with model size:** 3B model exhibits largest reduction in post-finetuning test loss under SPT

Scaling with Model Size

As capacity increases, so does the **relative advantage** of integrating domain data during pretraining

Domain Test Loss vs Model Size



NPT (δ=0%)	SPT (δ=2%)
Higher loss	Lower loss

INTERPRETATION

Model Size: The Overfitting Interpretation

Why Larger Models Benefit More

Greater Capacity

Larger models have **greater representational capacity**

More Prone to Memorization

Therefore **more prone to memorizing** limited domain corpus during finetuning

SPT's Protection

SPT's regularization effect **becomes more valuable**

Implications

1 Large-Model Regime

SPT becomes **increasingly effective** in the large-model regime

2 Current Trend

As models get larger (current trend), **SPT advantage grows**

3 Practical Impact

For deployed large models, SPT offers **even greater benefits**

Benefits of SPT amplify with model scale

The larger the model, the more important early domain integration becomes

FACTOR 4

Pretraining Compute Budget

Optimal Mixture Depends on Budget

The **optimal mixture fraction δ** depends on the available pretraining compute.

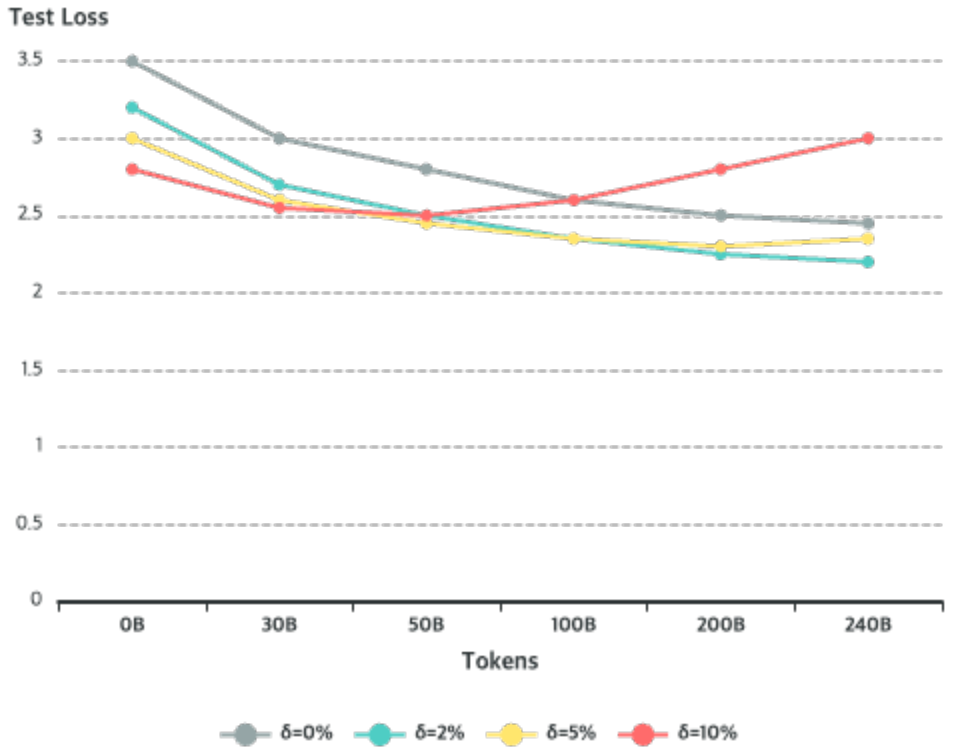
Study Design:

Track domain test loss throughout training for mixture fractions ranging from **0% to 10%** on MusicPile

The Tradeoff

Larger δ% optimal at **shorter scales**, but threshold exists where test loss **degrades** due to excessive repetition

Test Loss vs Tokens by Mixture



<30B tokens
δ=10% best

>200B tokens
δ=2% best

PRACTICAL GUIDANCE

Compute Budget: Practical Guidance

Small

Small Budget

<30B tokens

Optimal Strategy:

- ✓ Larger mixtures (δ=10%) achieve **lowest test loss**
- Accelerate learning of **domain structure**

Use higher δ%

Medium

Medium Budget

~50B tokens

Transition Point:

- ⇄ δ=10% curve **begins to degrade**
- ↑ Moderate mixtures (δ=5%) **continue improving**

Shift to moderate δ%

Large

Large Budget

>200B tokens

Optimal Strategy:

- ★ Best performance from **smaller mixtures** (δ=2-5%)
- Avoid **overfitting** from excessive repetition

Use lower δ%

★ Crucial: Regardless of budget, SPT with appropriate δ outperforms NPT. Treat δ as function of training horizon: higher for short runs, lower for long runs.

08

REPLAY vs SPT

Is replay a substitute for specialized pretraining?

REPLAY

Replay: A Common Anti-Forgetting Strategy

What is Replay?

Replay is commonly used during finetuning to mitigate forgetting by **mixing previously seen data back into training.**

The Core Question:

If replay already reintroduces general data during finetuning, does it have a similar regularizing effect as SPT? Or does **early exposure through SPT still outperform NPT?**

Experimental Setup

Compare **SPT→FT** against **NPT→FT** with replay-based FT mixing MusicPile with Dolma replay

Configuration

Replay Rates

Evaluate **{0%, 10%, 20%}** replay

Tuning

Tune **learning rate separately** for each setting

Reporting

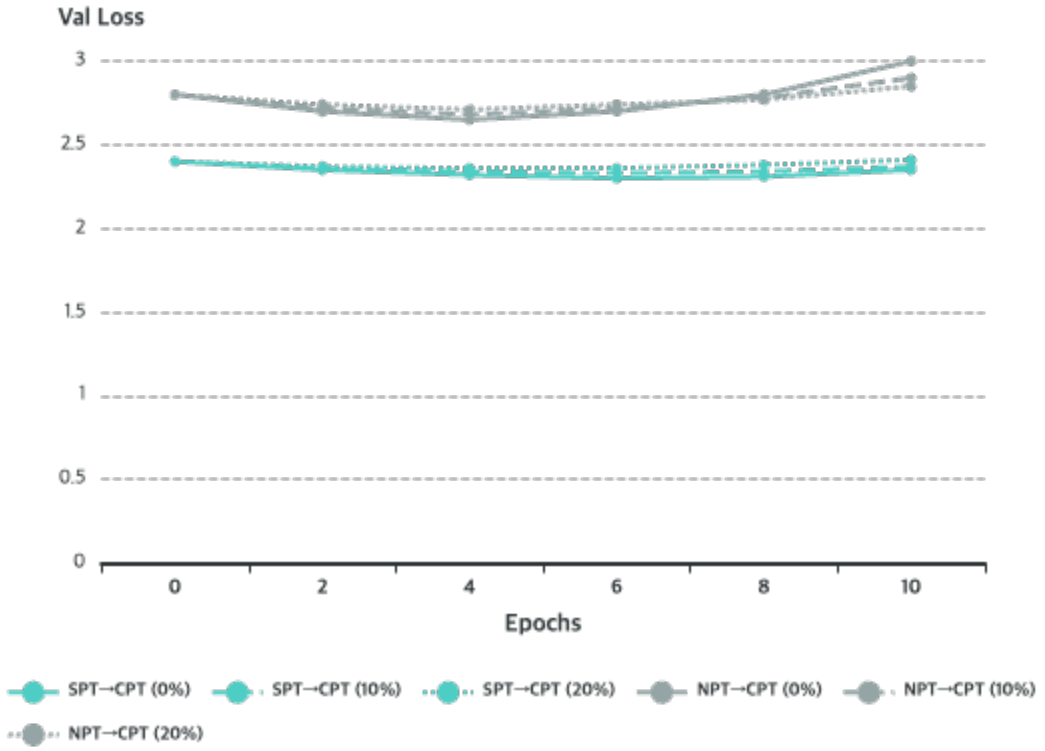
Report **MusicPile test loss** across finetuning trajectory

Goal: Test if replay substitutes for early domain exposure

RESULTS

Replay Results: SPT Advantage Persists

Validation Loss Trajectories



SPT → CPT
Lower loss

NPT → CPT
Higher loss

Key Findings

1 Consistent SPT Advantage

Across **all replay settings**, SPT→FT consistently achieves lower domain test loss than NPT→FT

2 Replay Helps NPT, But...

10% replay **helps NPT**, but NPT→FT falls **well short** of SPT→FT with no replay

3 Qualitatively Different Effects

Two forms of mixing induce **qualitatively different effects**

★ Core thesis reinforced: when domain data is seen matters

09

SCALING LAWS

Predicting optimal domain-data repetition

THE CHALLENGE

The Prediction Challenge

The Problem

Specialized pretrained models see **multiple repetitions** of finetuning data starting from pretraining.

The Dependency:

Optimal mixture percentage δ depends on the **pretraining compute budget**

The Tradeoff:

Larger δ% optimal at **shorter scales**, but threshold exists where test loss **degrades** from excessive repetition

Core Question

Can we predict domain test loss across specialized pretraining and finetuning as a function of δ?

Why It's Challenging

1 Model Overfitting Regime

Must explicitly **model overfitting regime**, which standard power law cannot anticipate

2 Post-Finetuning Prediction

Unclear how to **predict scaling laws post-finetuning**

3 Complex Interactions

Must capture interactions between **pretraining and finetuning**

Need new approach to model overfitting scaling laws

SOLUTION

Two-Step Modeling Approach

The Decomposition

Divide modeling overfitting scaling laws into **two steps:**

1 Model SPT Scaling

Separately model how domain **train and test losses scale** during SPT as function of δ

+

2 Model Finetuning Effect

Measure **difference in test loss** before and after finetuning, which scales reliably with pretraining tokens

✓ Test loss improves up to certain repeats, then overfits

Previous Work Limitations

Kaplan et al. (2020); Goyal et al. (2024); Muennighoff et al. (2023):

Express overfitting as power law with exponent modeled by **decaying function of repetitions**

Problems:

- ✗ Introduces **nested nonlinearities**
- ✗ Difficult to fit from **limited data**
- ✗ Hard to **extrapolate** beyond range

Our Alternative:

Simple decomposition that separately models training loss and train-test gap as power laws

DECOMPOSITION

The Decomposition Solution

Simple Alternative Model

Instead of fitting single power law, **decompose test loss** into training loss and train-test gap:

↓ Training Loss

Modeled using **usual power law** with negative exponent

Decreases monotonically

+

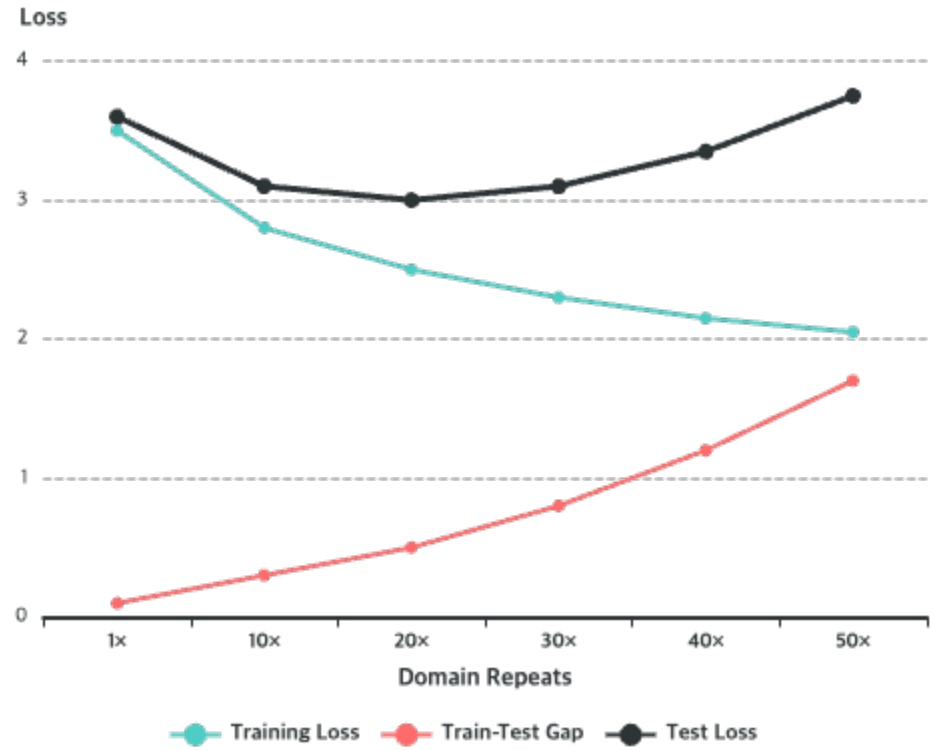
↑ Train-Test Gap

Monotonically increases, modeled with positive exponent

Represents overfitting

= Test Loss = Training Loss + Train-Test Gap

Illustration Across SPT Runs



Training	Gap	Test
Decreases	Increases	Non-monotonic

FORMULATION

Scaling Law Formulation

Core Equations

Training Loss:

$L_{train}(T,\delta) = A_{train} \cdot T_{btrain(\delta)} + C_{train}(\delta)$

Train-Test Gap:

$L_{gap}(T,\delta) = A_{gap}(\delta) \cdot T_{bgap(\delta)}$

Test Loss:

$L_{test}(T,\delta) = L_{train}(T,\delta) + L_{gap}(T,\delta)$

Coefficient Functions

$b_x(\delta) = \delta \cdot b_{x,s} + (1-\delta) \cdot b_{x,g}$

$A_{gap}(\delta) = \alpha_1 \cdot \delta_{\alpha_2} \cdot \exp(\alpha_3 \cdot \delta)$

$C_{train}(\delta) = \kappa_0 - \kappa_1 \cdot \log(\delta+\kappa_2) - \kappa_3 \cdot \delta$

Design Choices

A_{train}

Fixed to be a **constant**

$A_{gap}(\delta)$

Modeled using **Gamma kernel function**

$C_{train}(\delta)$

Use **log-linear expression** (best fits data)

All coefficients modeled as functions of δ

INTERPRETATION

Interpreting the Exponent

Interpretable Form

The exponent has an **interpretable form** that captures the tradeoff between general and domain data utility:

Rate of decrease modeled by:

- **General pretraining data** (b_g)
- **Specialized domain data** (b_s)

Both have negative utility (both improve loss)

Key Insight

$b_{train}(\delta)$ is **strictly negative**, indicating training loss decreases monotonically throughout pretraining

Training vs Gap Exponents

Training Loss Exponent

Both exponents **negative:** $b_g, b_s < 0$

$|b_g| < |b_s|$: domain loss goes down slower for smaller δ

Train-Test Gap Exponent

Tradeoff between **$b_g < 0$** and **$b_s > 0$**

Higher δ accelerates overfitting

Sublinear Growth

b_{gap} remains **below 1** across all δ up to 10%

Overfitting grows sublinearly → SPT sustains effectiveness

POST-FINETUNING

Predicting Test Loss After Finetuning

The Challenge

Now that we have accurate model for domain-specific test loss as function of pretraining compute, turn attention to **predicting test loss after finetuning.**

Direct modeling is challenging:

Entangles contributions of **pretraining and finetuning**

The Solution

Leverage existing pretraining loss model and **separately characterize marginal effect of finetuning**

The Delta Test

Definition:

$\Delta\ell_{test} = \ell_{test}(\theta_{PT}) - \ell_{test}(\theta_{PT+FT})$

Change in domain test loss after finetuning

Interpretation:

Positive value = finetuning **reduced** test loss

Always > 0 with early stopping

Power Law Relationship:

$\Delta\ell_{test} = a \cdot T_b + c$

Follows remarkably consistent power law

POWER LAW

The Delta Test Power Law

Empirical Observation

Empirically observe that change in test loss follows **remarkably consistent power-law relationship** as function of pretraining steps T:

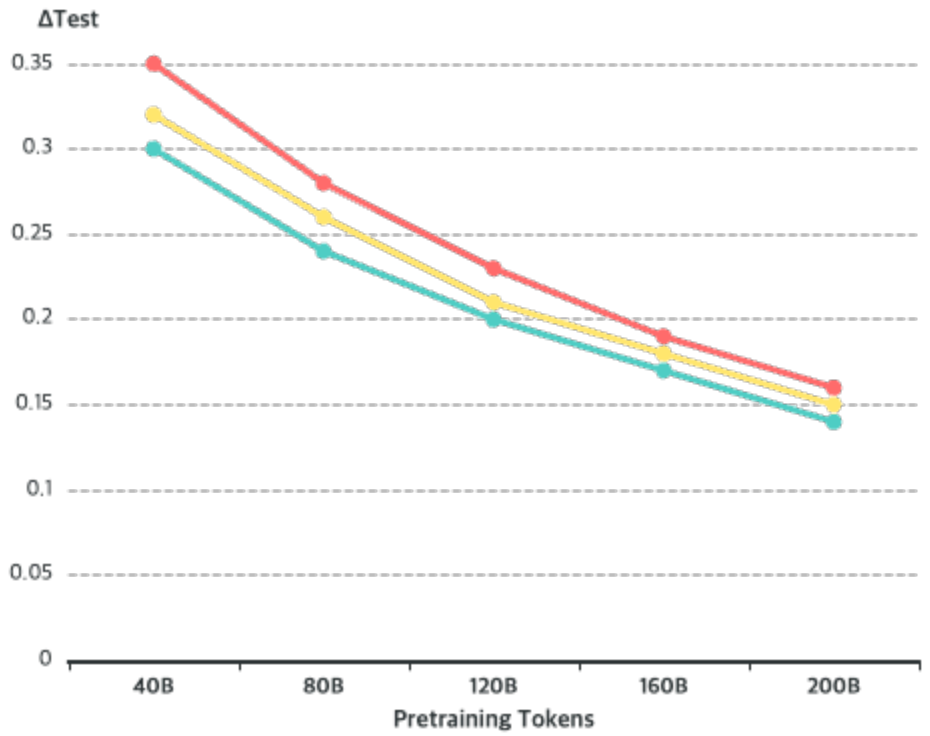
$$\Delta \ell_{test} = a \cdot T_b + c$$

Intuition:

Captures how benefit of finetuning **varies depending on stage** of pretraining at which model is finetuned

✓ Illustrated for all domains in Appendix D

ΔTest vs Pretraining Tokens



Power law relationship observed across all three domains

FORECASTING

Forecasting Example 1: Predicting 10% SPT Curve

The Approach

1 Model Formulation

Model domain test loss as **sum of two powers** with exponent set to $(1-\delta)\cdot b_g + \delta\cdot b_s$

2 Learn from Limited Data

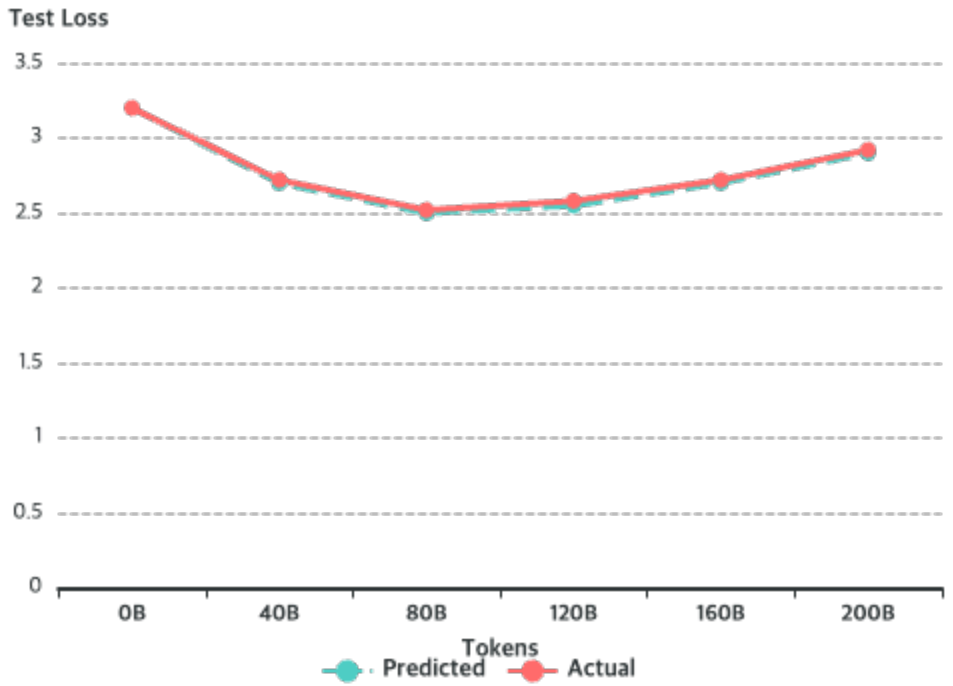
Learn b_g, b_s, and other variables using only $\delta \in [0, 5]$

3 Extrapolate

Extrapolate **full domain loss curve** of SPT for 200B tokens at δ=10%

Goal: Predict without running full 10% training

Prediction vs Actual



Forecast tracks observed values closely

Correctly predicts overfitting at ~70B tokens

FORECASTING

Forecasting Example 2: Post-FT Loss Extrapolation

The Approach

1 Combine Laws

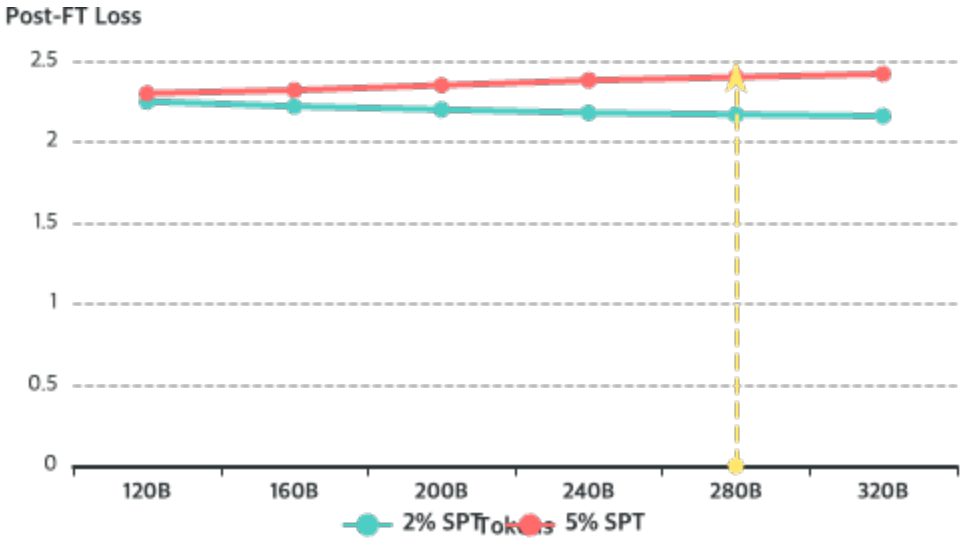
Combine **pretraining scaling laws** with **$\Delta\ell$test power law**

2 Extrapolate

Fully extrapolate **post-finetuning loss** beyond 120B tokens for $\delta \in [0, 5\%]$

Goal: Predict optimal mixture without full training

Prediction: 2% vs 5% SPT



Correctly predicts 2% surpasses 5% at ~280B

Consistent with full runs in Figure 9

Practical Impact: Scaling laws eliminate need for expensive sweeps. Small number of pilot runs suffices to fit expressions and select right SPT configuration.

10

THE FALLACY EXPLAINED

Why finetuning alone is often the wrong choice

THE FALLACY

The Conventional Wisdom

Standard Approach

Standard approach to domain adaptation: **finetune large general-purpose model.**

Why It Seems Right:

Appears **cheap because it avoids cost of pretraining**

The Hidden Cost:

This accounting **ignores inference** costs

The Reality

A **1B SPT model** costs more upfront than finetuning **3B model**, but **3× smaller model is cheaper to serve**

Total Cost of Ownership

Training Cost

1B SPT: **Higher** upfront cost
3B NPT→FT: **Lower** upfront cost

Inference Cost

1B SPT: **3× cheaper** to serve
3B NPT→FT: **3× more expensive**

Break-Even Point

Arrives after **~1 trillion inference tokens**

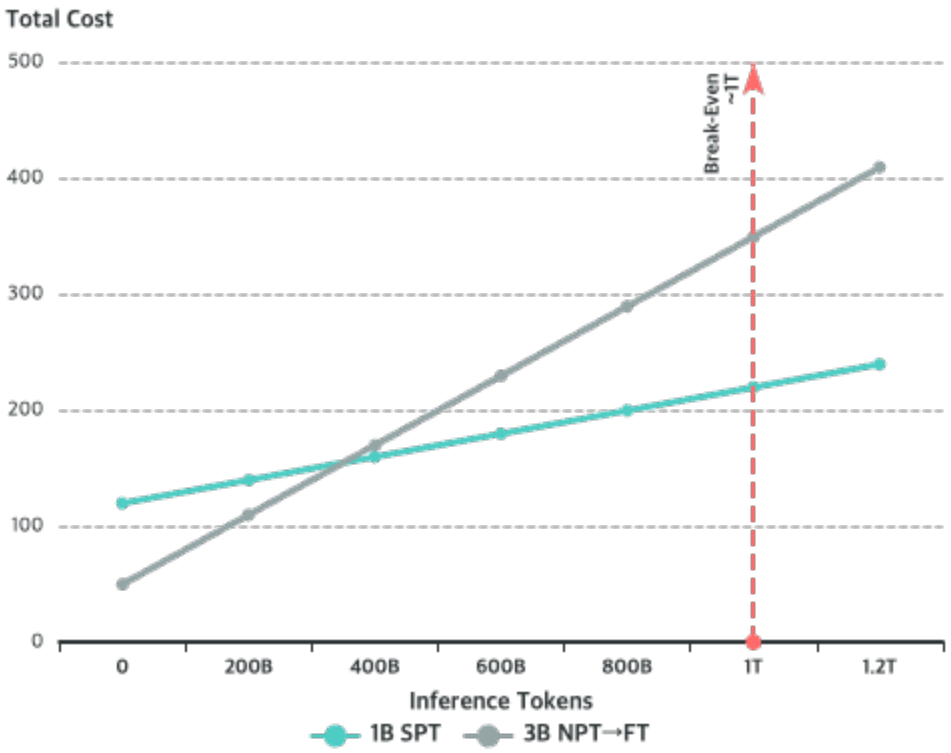
After which SPT saves both compute and money

For deployed models, inference dominates total energy consumption

BREAK-EVEN

The Break-Even Analysis

Cumulative Cost Over Time



1B SPT
Cheaper long-term

3B NPT→FT
Cheaper short-term

The Math

Training Cost:

SPT: **Higher** (pretraining + FT)
NPT→FT: **Lower** (FT only)

Per-Inference Cost:

1B: **1 unit**
3B: **3 units** (3× more expensive)

Break-Even:

After **~1T inference tokens**

SPT training cost = Inference savings

Fewer parameters = Lower carbon footprint

NUANCE

Nuanced Understanding: Not Always Pretrain

Important Clarification

Importantly, the finetuner's fallacy is **NOT**:

❌ "Fine-tuning is always enough"

❌ "Always pretrain your model"

The Truth:

Real systems occupy **different points on a continuum**, and the right strategy depends on multiple factors

⚖ Context matters: data, domain, constraints

Three Key Decision Factors

1 Domain Data Amount

How much **domain-specific data** do you control?

More data → Full SPT preferred

2 Domain Distance

How far does your domain sit from **web text**?

Further → Larger SPT benefit

3 Serving Constraints

What are your **scale and latency requirements**?

Stricter → SPT more attractive

GUIDANCE

Navigating the Tradeoffs

Key Decision Factors

Domain Similarity

Lower similarity to pretraining corpus → **Larger SPT benefit**

Domain Dataset Size

Larger datasets → **Full SPT preferred**
Smaller datasets → **SCPT may be better**

Model Size

Larger models → **SPT advantage increases**

Pretraining Budget

Shorter budgets → **Higher δ%**
Longer budgets → **Lower δ%**

Scaling Laws as Guide

Pilot Runs

Let practitioners navigate tradeoff from **small number of pilot runs**

No Exhaustive Search

Eliminates need for **expensive sweeps** over mixture fractions and training horizons

Predictive Power

Fit expressions and **select right SPT configuration** for given compute budget

Overfitting scaling laws make SPT both effective and predictable

11

RELATED WORK

Positioning within the broader research landscape

FORGETTING & REPLAY

Catastrophic Forgetting and Replay

Historical Context

Catastrophic forgetting studied since McCloskey & Cohen (1989): learning new information overwrites previously acquired knowledge.

Gururangan et al. (2020): Continued pretraining improves downstream performance but can degrade general capabilities

Luo et al. (2023): Forgetting intensifies as model scale increases from 1B to 7B parameters

Gupta et al. (2023): How to "rewarm" learning rate schedules for continual pretraining

Shi et al. (2024): Comprehensive survey of continual learning strategies

Replay: Most Widely Adopted Defense

Parmar et al. (2024)

Recipe for continued pretraining mixing general and domain data to mitigate forgetting

Blakeney et al. (2024)

Domain upsampling at end of training can yield performance gains

Kotha & Liang (2026)

Replaying generic pretraining data during finetuning prevents forgetting AND improves target-task performance (up to 2.06× data efficiency)

Replay helps more when less target data in pretraining mix

MEASUREMENT

Measurement of Forgetting

Fine-Grained Analysis

Harmon et al. (2025)

Sample-wise metrics revealing that **large per-example forgetting** can hide beneath stable aggregate accuracy

Thede et al. (2026)

CapTrack: Capability-centric framework showing post-training forgetting extends beyond factual knowledge to multilingual robustness, instruction following, and calibration

Our work: Replay not substitute for early domain exposure

Connection to Our Work

Consistent Finding

Our work (§4) shows that replay during continued pretraining is **not a substitute** for early domain exposure: SPT's gains persist across all replay settings

Broader Message

Consistent with broader message: **when data appears** in training pipeline matters as much as **whether it appears**

Complementary

Kotha & Liang (2026) find replay helps more when less target data in pretraining mix → **directly complements** our finding that SPT's advantage is largest when domain is underrepresented

SCALING LAWS

Scaling Laws for Data Mixing

Foundational Work

Kaplan et al. (2020)

Established first scaling laws relating **model size, dataset size, and compute** to language modeling loss

Muennighoff et al. (2023)

Extended to **data-constrained regimes** where tokens must be repeated. Multiple epochs beneficial up to point but eventually yield diminishing returns

Don't model overfitting stage we observe

Data mixing laws predict loss from mixture proportions

Data Mixing Laws

Ye et al. (2024); Goyal et al. (2024)

Propose data mixing laws that **predict loss as function of domain mixture proportions**, enabling optimization from small pilot runs

Que et al. (2024)

Derive **domain-specific continual pretraining scaling law** predicting optimal mixture ratio as function of model size and data budget

Gap

None address **non-monotonic test loss trajectory** from heavy repetition of small domain corpus

OUR CONTRIBUTION

Our Contribution to Scaling Laws

Novel Decomposition

Our overfitting scaling laws **complement** prior efforts by:

Separate Modeling

Separately model **training loss and train-test gap** as power laws with opposing exponents

Capture Non-Monotonicity

Captures **non-monotonic test loss trajectory** from heavy repetition of small domain corpus

Regime not addressed by prior formulations

★ Practical: Enable extrapolation from pilot runs

Key Innovations

1 Two-Power Decomposition

Test loss = Training loss (negative exponent) + Train-test gap (positive exponent)

2 Parametrize by δ

All coefficients as **functions of mixture fraction δ**

3 Extrapolation Capability

Enable extrapolation across **both mixture percentages and training horizons** from small set of pilot runs

Makes specialized pretraining both effective and predictable

SYNERGY

Pretraining vs Post-Training Synergy

Capability-Oriented View

A **capability-oriented view** of the pretraining-vs-post-training boundary is emerging across several subfields:

Safety Alignment:
Maini et al. (2025), Sam et al. (2025): Behaviors acquired during pretraining are **harder to remove** via post-training

Multilingual Settings:
Longpre et al. (2025): Cross-language transfer during pretraining improves low-resource language performance **more effectively**

Post-Training Literature:
Chu et al. (2025): RL with outcome-based rewards **generalizes** whereas SFT **memorizes**

Common thread: When data appears matters

Common Thread: Memorization vs Generalization

The Core Insight

The **manner in which data is introduced**, not just its quantity, determines whether the model **memorizes or generalizes**

SPT Operationalizes This

By **interleaving domain tokens among general data**, no single batch is dominated by scarce domain corpus, producing **regularization effect** analogous to RL's on-policy sampling

Connection

Shenfeld et al. (2025) formalize as **RL's Razor**: on-policy RL implicitly biased toward policy closest in KL divergence to base model

Lai et al. (2025) confirm this mitigates forgetting

UNIFYING PRINCIPLE

Common Thread: Memorization vs Generalization

The Unifying Principle

The manner in which data is introduced, not just its quantity, determines whether the model memorizes or generalizes

Memorization
Concentrated exposure

Generalization
Diffused exposure

SPT's Role

1 Operationalization

SPT **operationalizes this principle** at the pretraining stage

2 Mechanism

By **interleaving domain tokens among general data,** no single batch is dominated by scarce domain corpus

3 Result

Produces **regularization effect** analogous to RL's on-policy sampling

SPT connects to broader understanding of training dynamics

INDUSTRY

Specialized Pretraining in Industry

BloombergGPT

Approach:

50B-parameter model on ~700B tokens combining 363B-token proprietary finance corpus with general data

Finance-aware model outperforms task-specific finetuned models

Character.AI

Approach:

Operates both **large custom conversational models trained from scratch** AND aggressively finetuned open-source models using SFT, DPO, and RL

Different strategies for different use cases

Cursor Composer

Approach:

RL-based post-training atop existing coding backbone rather than training from scratch

Opposite direction: post-training focus

Industry practice lies on a continuum indexed by data scale, domain shift, and deployment constraints

IMPLICATIONS

Industry Implications

Cursor's Future Challenge

As Cursor pursues increasingly **agentic capabilities**, it may encounter the finetuner's fallacy anew

Why:

Finetuning a base model that **lacks agentic traces** will eventually yield diminishing returns

Not "always finetune" or "always pretrain"

Continuum of Strategies

1. **Domain Data Amount**

   How much **domain-specific data** they control

2. **Domain Distance**

   How far their domain is from **web-scale distributions**

3. **Deployment Constraints**

   Scale and latency requirements

Goal: Move from anecdotes to systematic characterization

12

CONCLUSIONS

Key takeaways and future directions

KEY TAKEAWAYS

Core Conclusions

The Broader Lesson

Scarce and specialized domain data should NOT be treated as a final-stage resource

✓ Mixing even **1-5% domain data** into pretraining consistently outperformed standard pipeline

✓ Gains persisted even under **replay-based continued pretraining**

✓ **When** domain data enters training pipeline has lasting impact

↘ Pretraining costs falling → Case for early integration strengthens

Paradigm Shift

✕ **Away From:**

Post-training patches applied to generic checkpoints

↓

✓ **Toward:**

Natively specialized models that incorporate domain knowledge from the start

For Organizations

As organizations increasingly seek to deploy models tailored to **proprietary domains**, these findings point toward this shift

RECOMMENDATIONS

Practical Recommendations

Five Key Recommendations

1. **Incorporate Early**
   Incorporate domain data as **early as possible**
2. **Start with 1-5%**
   Use **1-5% mixture fraction** as starting point
3. **Adjust δ Based on Context**
   Adjust based on **compute budget and dataset size**
4. **Consider SCPT for Small Data**
   For **very small domain datasets**
5. **Use Scaling Laws**
   Guide configuration selection from **pilot runs**

Decision Framework

Domain Similarity:
Low similarity → **Larger SPT benefit** → Prioritize SPT

Dataset Size:
Large (300M+) → **Full SPT**; Small (3M-30M) → **SCPT**

Compute Budget:
Short → **Higher δ%**; Long → **Lower δ%**

Model Size:
Larger models → **SPT advantage increases**

Scaling laws make SPT predictable from pilot runs

FUTURE WORK

Future Research Directions

1

Mechanism Understanding

Precise mechanism behind asymmetry between **diffuse domain exposure during pretraining** versus **general-data replay during finetuning** remains open

Why do they induce qualitatively different effects?

2

Scaling Law Extensions

Extend scaling laws to **more diverse domains** and **model architectures**

Test generalizability across settings

3

Data Curation Synergy

Understand interaction between SPT and **other data curation strategies** like synthetic augmentation

Can they be combined effectively?

4

Multimodal Settings

Investigate SPT in **multimodal settings** (vision-language, audio-text)

Does the same principle apply?

5

Better Metrics

Develop more sophisticated **metrics for predicting SPT benefit** from distributional analysis

Current metrics have limitations

6

Adaptive Strategies

Explore **adaptive mixture strategies** that adjust δ during training

Dynamic vs fixed mixtures

THANK YOU

The Finetuner's Fallacy

To get the most out of domain data,

incorporate it as early in training as possible.

DatologyAI Team

Research Paper: March 2026